

SpecBot

Project Team

Muhammad Abdur Rafey	21I-0705
Ayra Alamdar	21I-2968
Husnain Ali	21I-0542

Session 2021-2025

Supervised by

Dr. Arshad Islam



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

June, 2024

Contents

1	Introduction	1
1.1	Problem Statement	2
1.2	Scope	2
1.3	Modules	3
1.3.1	Hardware Specification	3
1.3.2	Static Code Analysis	3
1.3.3	Data Dependency Analysis	4
1.3.4	Thread Enhancement	4
1.3.5	Parallelization Algorithm	4
1.3.6	VSCode Integration	4
1.4	User Classes and Characteristics	5
2	Project Requirements	7
2.1	Use Case Diagram	7
2.2	Event Response Table	8
2.3	Functional Requirements	9
2.3.1	Hardware Specification Module	9
2.3.2	Static Code Analysis Module	9
2.3.3	Data Dependency Analysis Module	9
2.3.4	Thread Enhancement Module	9
2.3.5	Parallelization Algorithm Module	10
2.3.6	Deployment and Integration Module	10
2.4	Non-Functional Requirements	10
2.4.1	Reliability	10
2.4.2	Usability	10
2.4.3	Performance	11
2.4.4	Security	11
2.5	Domain Model	12
3	System Overview	13
3.0.1	Design	13
3.0.2	Background	13
3.1	Architectural Design	14

3.2	Design Models	15
3.2.1	Activity Model	16
3.2.2	Data Flow Diagram	17
3.2.2.1	Level 0 (Context Diagram)	17
3.2.2.2	Level 1 (Main Process)	18
3.2.2.3	Level 2 (Sub-processes)	19
3.2.2.4	Level 3 (Detailed Processes)	20
3.2.3	System Level Sequence Diagrams	21
3.2.3.1	Code Analysis Module	21
3.2.3.2	Code Deployment Module	22
3.2.3.3	Parallelization Decision Module	23
3.2.3.4	Thread Optimization Module	24
3.2.3.5	Parinomo Engine Module	25
3.2.4	State Transition Diagram	26
3.3	Data Design	27
3.3.1	Data Structures Utilized	27
3.3.1.1	Abstract Syntax Tree	27
3.3.1.2	Data Dependency Graph	27
3.3.2	Data Storage and Communication	27
3.3.2.1	JSON Schema	27
3.3.2.2	Authentication	29
	References	31

List of Figures

1.1	Workflow of SpecBot	2
2.1	Use Case Diagram	7
2.2	Domain Model of SpecBot	12
3.1	Architecture Diagram	14
3.2	Activity Model	16
3.3	Level 0	17
3.4	Level 1	18
3.5	Level 2	19
3.6	Level 3	20
3.7	Code Analysis Module SSD	21
3.8	Code Deployment Module	22
3.9	Parallelization Decision Module	23
3.10	Thread Optimization Module	24
3.11	Parinomo Engine Module	25
3.12	State Transition Diagram	26
3.13	Frontend Schema	28
3.14	Backend Server Response	28

List of Tables

1.1	User Groups and Their Benefits for SpecBot	5
2.1	Event Response Table of SpecBot	8

Chapter 1

Introduction

In the ever-changing field of software development, code execution efficiency has become more and more important. Software that can fully utilize these resources is becoming more and more necessary as hardware capabilities continue to develop [3]. The goal of the SpecBot project is to close the gap between sequential code and contemporary computer systems' parallel processing capabilities.

The automated conversion of sequential code to parallel code is the main goal of this project, with a focus on performance optimization according to hardware setups. SpecBot aims to improve the speed and efficiency of computationally demanding processes by examining the code structure, finding parallelizable portions, and taking the underlying hardware specs into account [5].

This approach not only improves the performance of existing code but also promotes the development of more hardware-aware software solutions in the future.[6]

This document specifies the requirements for SpecBot, an automated tool that converts sequential code to parallel code with performance optimization based on specific hardware specifications. It is intended for the following readers:

- Developers working on implementing SpecBot
- Project managers overseeing the development process
- Testers who will verify SpecBot's functionality
- Documentation writers creating user guides and technical documentation
- Potential users of SpecBot, including software engineers and researchers

1.1 Problem Statement

Aiming to improve the excess capacity that modern multi-core CPUs offer for parallel processing, SpecBot undertakes a computing task that calls for high performance. Many software engineers continue to write sequential code or employ such tools as code generation that do not take full advantage of parallel processing, and so performance for these tasks is still mediocre. Even when the idea of parallelizing something is there, for some reason it always neglects a certain aspect of the hardware configuration, making the idea inefficient. SpecBot provides the necessary tools to transform the sequential processing operating code into parallel processing operating code according to the user's hardware needs. This solution narrows the performance gap between the latest advances in hardware technology and necessary software performance in order to enhance application performance in high end computing dependent industries across the board.

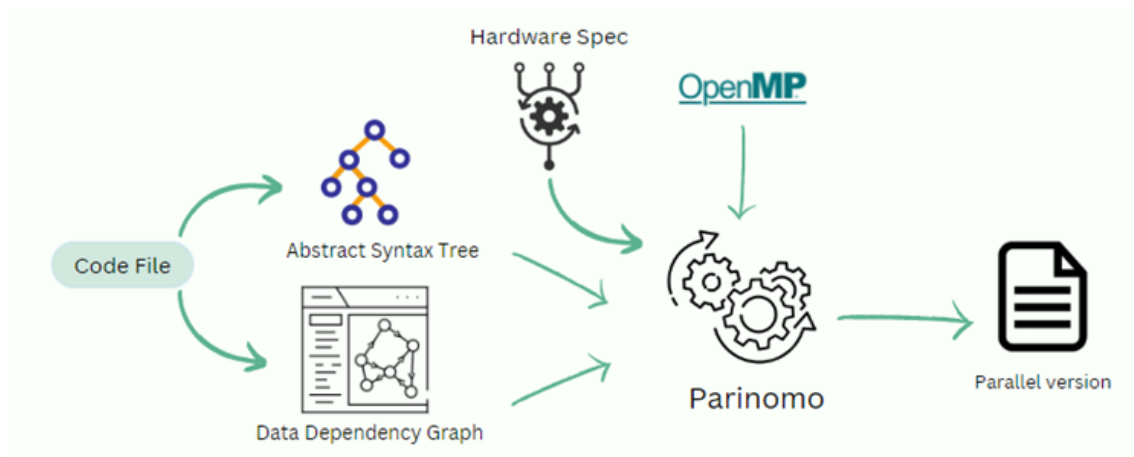


Figure 1.1: Workflow of SpecBot

1.2 Scope

SpecBot's scope encompasses the development of an automated tool for converting sequential C/C++ code to parallel code, with a focus on performance optimization based on specific hardware specifications. The project will be available as both a Visual Studio Code extension and a standalone website. Key functionalities include:

- Analyzing sequential C/C++ code to identify parallelizable segments
- Examining hardware specifications (core count, RAM, etc.)
- Identifying computationally intensive sections within the code

- Generating data dependency graphs to ensure safe parallelization
- Automating code parallelization based on the analyzed information
- Optimizing thread allocation considering data dependencies and hardware capabilities
- Assessing parallelized code performance before execution
- Supporting a wide range of applications, particularly those involving computationally intensive tasks
- Improving the performance of C/C++ code, which indirectly enhances the efficiency of other programming languages that rely on C/C++ libraries
- Providing flexibility to users through both VSCode extension and web-based interfaces

The project aims to make parallel programming more accessible to developers without expertise in this area while producing correct and efficient parallelized code.

1.3 Modules

Our project comprises several interconnected modules, each meticulously designed to address specific aspects of code parallelization. These modules work in concert to analyze, optimize, and transform sequential code into efficient parallel implementations. The functional requirements of our project are as follows:

1.3.1 Hardware Specification

1. Develops a web interface for user interaction
2. Implements system analysis for Unix-based systems
3. Utilizes JavaScript, Python, or Bash to extract critical system information (RAM, CPU cores, threads)

1.3.2 Static Code Analysis

1. Generates Abstract Syntax Trees (AST) to represent code structure
2. Implements function section creation for modular analysis (tentative feature)

1.3.3 Data Dependency Analysis

1. Manages memory allocation and access patterns
2. Implements tiling strategies for optimized data access
3. Develops algorithms for efficient iteration division[4]

1.3.4 Thread Enhancement

1. Designs algorithms for dynamic thread size determination
2. Implements per-loop thread optimization (tentative feature)
3. Develops mechanisms for runtime thread adjustment

1.3.5 Parallelization Algorithm

1. Integrates AST[7] with Data Dependency Graphs for comprehensive code understanding
2. Facilitates seamless library integration for parallel computing
3. Automates the insertion of pragma directives for parallelization

1.3.6 VSCode Integration

1. Outlines integration procedures for Visual Studio Code
2. Defines system requirements for smooth operation
3. Plans deployment strategies for wide accessibility

Each module is crucial in transforming sequential code into its optimal parallel form, considering both code structure and hardware capabilities.

1.4 User Classes and Characteristics

User Group	Description
High-Performance Computing Centers	These centers use SpecBot to speed up big computational tasks by running them in parallel, making the work more efficient and reducing time spent.
Data Scientists	Data scientists rely on SpecBot to make machine learning training faster and use fewer resources by automatically spreading tasks across multiple processors.
Research Institutes	Research institutes use SpecBot to improve the performance of algorithms and simulations, which are crucial for scientific studies and AI research.
Programming Library Maintainers	People who manage C/C++ libraries use SpecBot to boost performance, which helps make other programming languages that depend on these libraries run faster and more efficiently.

Table 1.1: User Groups and Their Benefits for SpecBot

Project Requirements

Our use-case diagram helps to get an over-view on our user classes and their descriptions. It helps us to provide a more comprehensive understanding of how different users interact with the system and what actions they perform. Also, this integration enhances clarity by mapping each user role to its relevant use cases.



2.2 Event Response Table

An event table is used to explain how SpecBot responds to each event, providing a clearer understanding of its backend operations for stakeholders and external collaborators

Event	System State	Response
The user selects between system information retrieval or self-entering	The option between automatic retrieval and self-entering is displayed	Section for CPU, RAM, Threads and other information is displayed.
The user enters their code into the given code editor section	The webpage will display the system information and the code editor section	The code will be formatted in the editor
The user will select the button to trigger the code to be parallelized.	The webpage will display the system information and the code entered by the user.	The system information and code will be sent to the backend server to pass the code through the algorithm.
The serial code is sent to be parallelized.	A loading signal is displayed on the screen with updates on the point at which the strategy is applied.	• In the backend server AST and Data Dependency graph is generated and suitable number of threads are being selected. • The code is converted to parallel code and sent back to the front end.
Parallelized code returned to the front end.	The loading signal is come to an end and the returned code is displayed.	The profiler is used to show the difference between the original code and the parallelized code.
The user will observe the difference in serial and parallel code.	The profiler will highlight the difference between the user's code and the code given by the parallelizing strategies.	No response
The user will press the download code button	The profiler will display the difference in code and there will be an option to download the parallelized code.	The parallel code is placed in a file and will be downloaded in the user system. With an alert of downloaded.

Table 2.1: Event Response Table of SpecBot

2.3 Functional Requirements

This section describes the functional requirements of the SpecBot system, organized by module.

2.3.1 Hardware Specification Module

- **FR-1:** The system shall extract and analyze hardware information from Unix-based systems, including CPU cores, threads, and available RAM.
- **FR-2:** The system shall provide a mechanism to extract system information using the browser for web-based deployments.
- **FR-3:** The system shall store and utilize the extracted hardware information for optimizing code parallelization.

2.3.2 Static Code Analysis Module

- **FR-4:** The system shall generate Abstract Syntax Trees (AST) for input C/C++ code.
- **FR-5:** The system shall analyze the AST to identify regions suitable for parallelization.
- **FR-6:** The system shall determine whether to use static or dynamic AST analysis based on code complexity and size.

2.3.3 Data Dependency Analysis Module

- **FR-7:** The system shall identify shared data structures within the input code.
- **FR-8:** The system shall analyze dependencies between identified data structures.
- **FR-9:** The system shall highlight data structures that must be shared across threads for proper parallelization.

2.3.4 Thread Enhancement Module

- **FR-10:** The system shall develop algorithms to determine optimal thread size for each parallelizable loop.
- **FR-11:** The system shall implement mechanisms for dynamic thread adjustment during runtime.
- **FR-12:** The system shall investigate and implement methods for prioritizing process threads in the system queue.

2.3.5 Parallelization Algorithm Module

- **FR-13:** The system shall integrate libraries for parallel computing (OpenMP and OpenACC)[8].
- **FR-14:** The system shall identify code regions that are independent of data dependencies and suitable for parallelization.
- **FR-15:** The system shall manage data-dependent regions by determining efficient data-sharing mechanisms across threads.
- **FR-16:** The system shall select between OpenMP and OpenACC libraries based on code structure and hardware capabilities.

2.3.6 Deployment and Integration Module

- **FR-17:** The system shall be developed as a Visual Studio Code extension.
- **FR-18:** The system shall be deployed as a standalone website for broader accessibility.
- **FR-19:** The system shall benchmark the parallelized code output against competitor solutions for performance comparison.

2.4 Non-Functional Requirements

This section specifies the non-functional requirements for SpecBot.

2.4.1 Reliability

- The SpecBot system shall have a Mean Time Between Failures of at least 720 hours or 30 days of continuous operation.
- In the event of a system failure, SpecBot shall automatically save the current state of code analysis and notify the user, allowing for easy recovery and continuation of work.

2.4.2 Usability

- The SpecBot Visual Studio Code extension shall allow users to initiate code parallelization with no more than three mouse clicks or keyboard shortcuts.
- The web interface of SpecBot shall be designed to be intuitive, requiring no more than 10 minutes for a new user to understand and start using its basic features.

- SpecBot shall provide clear, concise error messages and suggestions for resolution when issues occur during the parallelization process.

2.4.3 Performance

- SpecBot shall complete the analysis and parallelization of code snippets up to 1000 lines within 30 seconds on a standard desktop computer (defined as having a quad-core processor and 8GB RAM).
- The system shall be capable of handling multiple simultaneous user requests on the web platform, with a maximum response time of 5 seconds for initiating the parallelization process.

2.4.4 Security

- All user-submitted code and generated parallel code shall be encrypted during transmission and storage using industry-standard encryption protocols (e.g., AES-256).
- The SpecBot system shall implement user authentication for the web platform, requiring strong passwords (minimum 12 characters, including uppercase, lowercase, numbers, and special characters).
- The system shall isolate and sandbox code execution environments to prevent potential security breaches from malicious code submissions.

2.5 Domain Model

The domain model represents an overview of the important notions and how they relate to one another in the SpecBot system. It shows entities like Code, HardwareSpec, AST, DataDependencyGraph, and ParallelizationStrategy demonstrating how these components cooperate. In this topology, one comprehends the interconnections among the various elements at the architecture level.

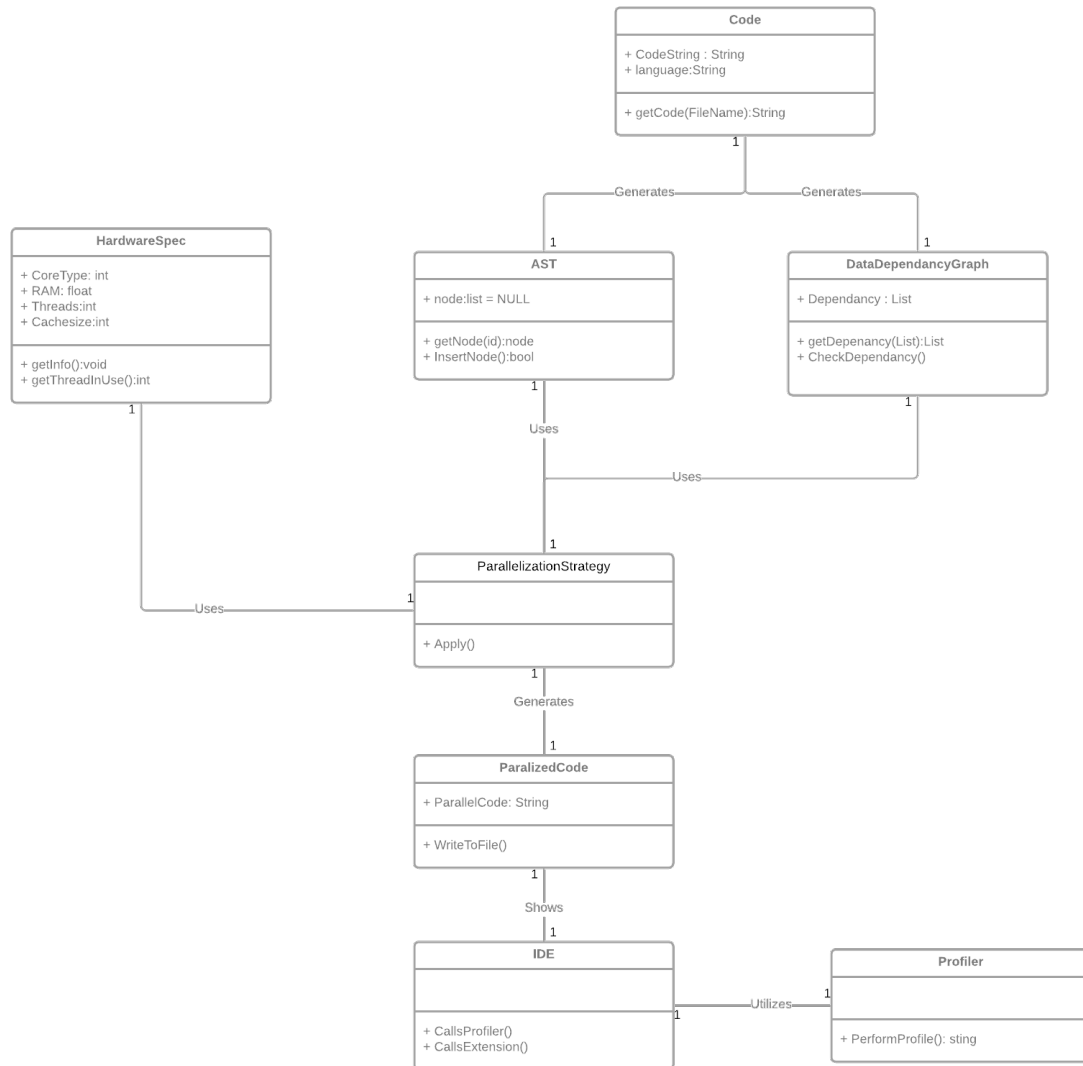


Figure 2.2: Domain Model of SpecBot

Chapter 3

System Overview

SpecBot is an automated tool that enhances software performance by converting sequential code to parallel code tailored for specific hardware configurations. It includes:

- **Hardware Analysis:** Gathers details about the user's CPU, RAM, and threading capabilities.
- **Code Analysis:** Generates Abstract Syntax Trees (AST) and Data Dependency Graphs from input code.
- **Parallelization Strategy:** Determines optimal parallelization based on code structure and hardware.
- **Code Transformation:** Applies techniques using libraries like OpenMP or OpenACC.
- **Performance Profiling:** Compares original and parallelized code performance.

3.0.1 Design

SpecBot has a modular architecture comprising:

- **Presentation Layer:** Features a Visual Studio Code extension and web interface.
- **Business Logic Layer:** Contains the Code Analyzer, Parallelization Engine, and Profiler.
- **Data Access Layer:** Manages file system interactions and hardware info retrieval.

3.0.2 Background

With the rise of multi-core processors, SpecBot addresses the challenge of utilizing these resources effectively. By automating the parallelization process, it aims to improve software performance and make parallel programming accessible to more developers across various industries.

3.1 Architectural Design

The architecture diagram below comprehensively displays the high-level design of the SpecBot system. The diagram above represents the main components of the system and the interactions among them, in essence, it forms a schematic of how the system components are arranged. This diagram assists the audience in the more powerful understanding of the overall system architecture, the aspects and conceptual areas, and coordination of different modules supporting the system objectives on the code optimization and parallelization.

The Architecture Diagram for SpecBot, represents the layers approach feature of the system, which contains the following main parts:

- **Presentation Layer:** This is the layer where end users interact with the System either through Visual Studio Code (VSC) as an extension or an online site by submitting the required information and gets the results.
- **Business Logic Layer:** This contains two sub-components:
 - *API Layer:* Represented by a castle-like icon, suggesting a secure gateway for data and functionality.
 - *Business Logic Layer:* Shown with gear icons, indicating where the core business rules and processes are implemented.
- **Backend Server:** This layer manages the AST and Data graph generation of the code and its conversion to the required parallel version.

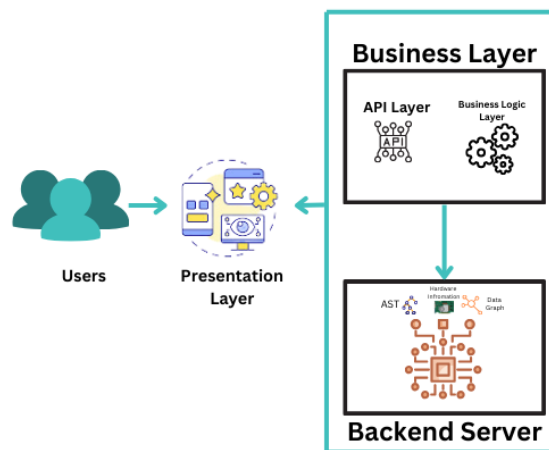


Figure 3.1: Architecture Diagram

3.2 Design Models

In contrast to the object-oriented approach, the procedural approach emphasizes functions over objects. For SpecBot, the **following design models** apply:

- **Activity Diagram:** Captures the step-by-step flow from hardware specification extraction to parallel code generation.
- **Data Flow Diagram:** Illustrates data movement through the system, from input code to AST generation and parallel code creation.
- **System-level Sequence Diagram:** Depicts interactions between system modules, like the code parser, thread optimizer, and parallelizer.
- **State Transition Diagram:** Models states of system modules managing backend processes such as thread allocation and AST generation.

We chose the **Procedural Approach** as it is ideal for SpecBot because it:

- **Streamlines Control Flow:** Facilitates a series of sequential operations, simplifying task management.
- **Optimizes Hardware Utilization:** Allows tight control over hardware interactions, crucial for thread allocation.
- **Ensures Efficient Execution:** Reduces overhead by focusing on direct function calls, resulting in faster execution.

This approach effectively transforms sequential code into optimized parallel code, making it suitable for high-performance computing environments.

3.2.1 Activity Model

The activity diagram shows the flow of activities and actions within the SpecBot system. It likely illustrates the step-by-step process of how the system analyzes code, performs parallelization, and generates output. This diagram helps visualize the overall workflow and decision points in the system's operation.

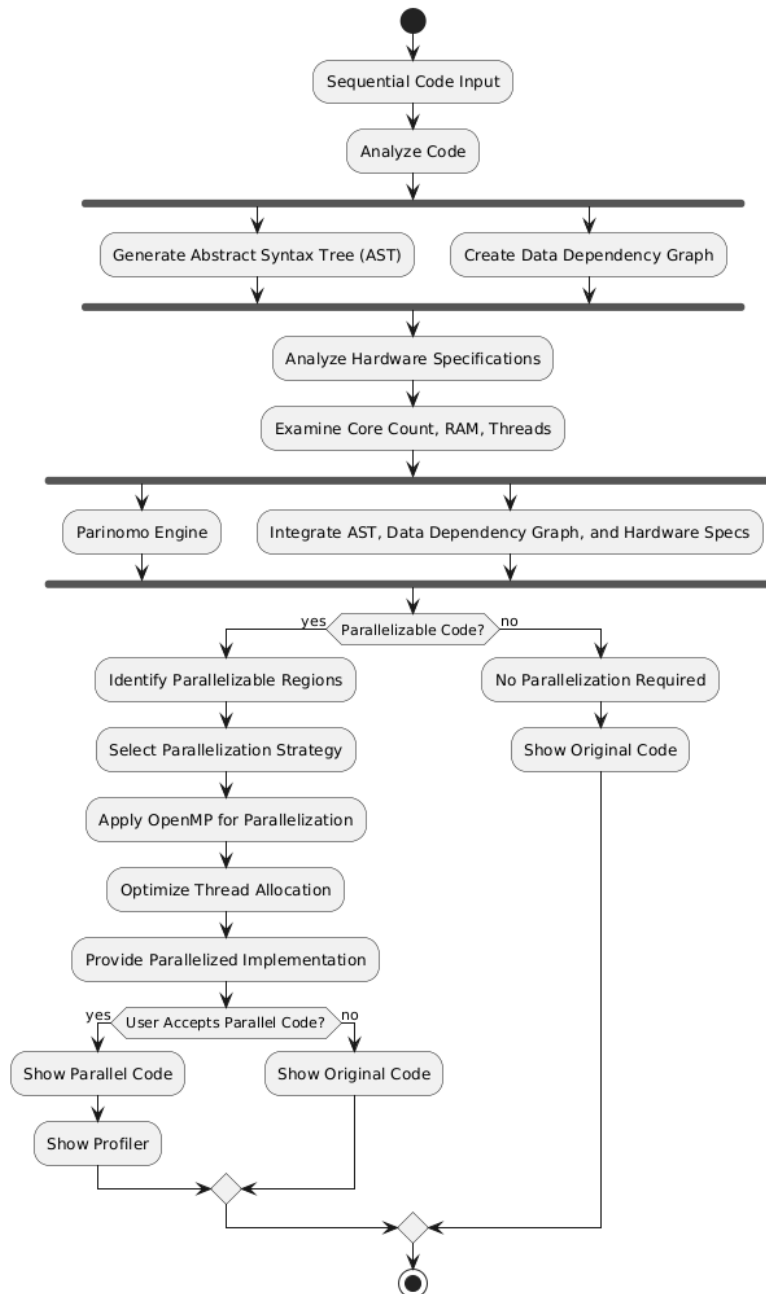


Figure 3.2: Activity Model

3.2.2 Data Flow Diagram

It would show how data moves through the system, from input code to AST generation, dependency analysis, and parallel code generation. It can be expanded into multiple levels to break down complex processes. The data flow diagram or our work flow diagram is as follows:

3.2.2.1 Level 0 (Context Diagram)

This is the highest-level view of the system. It shows the basic interaction between the user and the SpecBot system. The user provides serial code, and the system analyzes it to determine if parallelization is possible. The output is either parallelized code or feedback that the code cannot be optimized further.

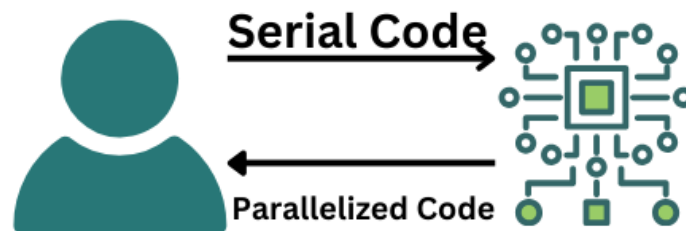


Figure 3.3: Level 0

3.2.2.2 Level 1 (Main Process)

This level provides more detail on the system's core processes. It shows that the system takes two inputs: the user's system information and the user's code. The code is processed to generate an Abstract Syntax Tree (AST) and a data dependency graph. These, along with the system information, are used to create a parallelization strategy. The strategy is then applied to generate parallelized code, which is returned to the user.

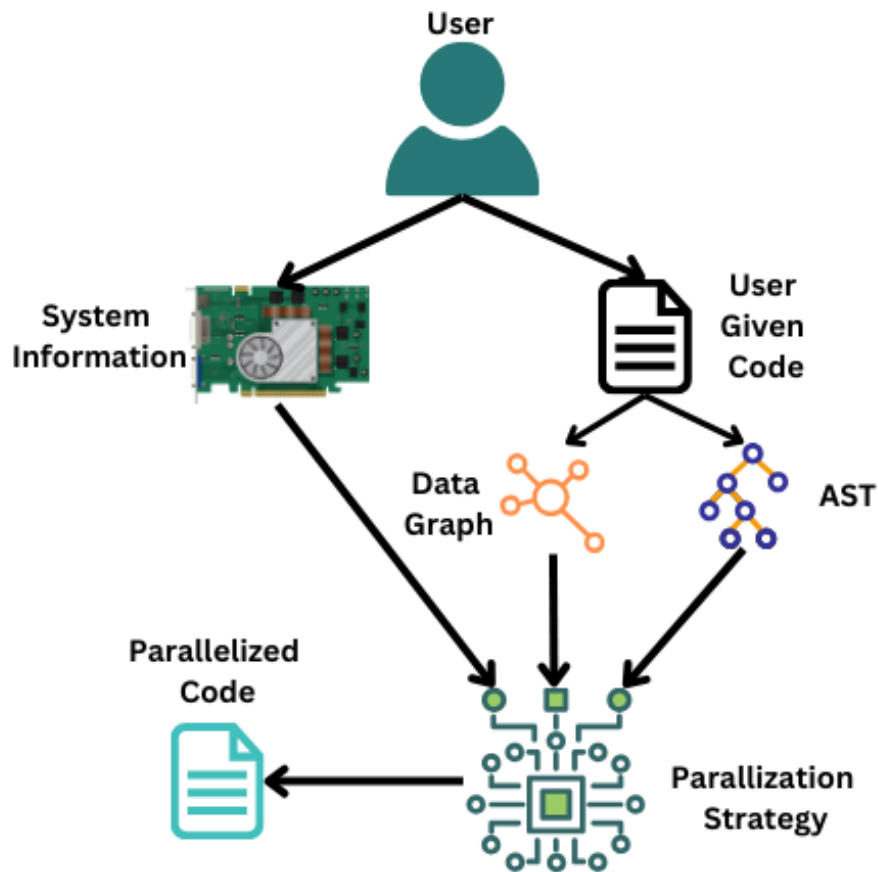


Figure 3.4: Level 1

3.2.2.3 Level 2 (Sub-processes)

This level maintains the flow from Level 1 but adds an important decision point. After the parallelization strategy is determined, the system decides whether to use OpenACC or OpenMP for the parallelization process. This decision is likely based on the code structure, the parallelization strategy, and the user's system capabilities. The chosen parallelization function (OpenMP or OpenACC) is then used to generate the final parallelized code.

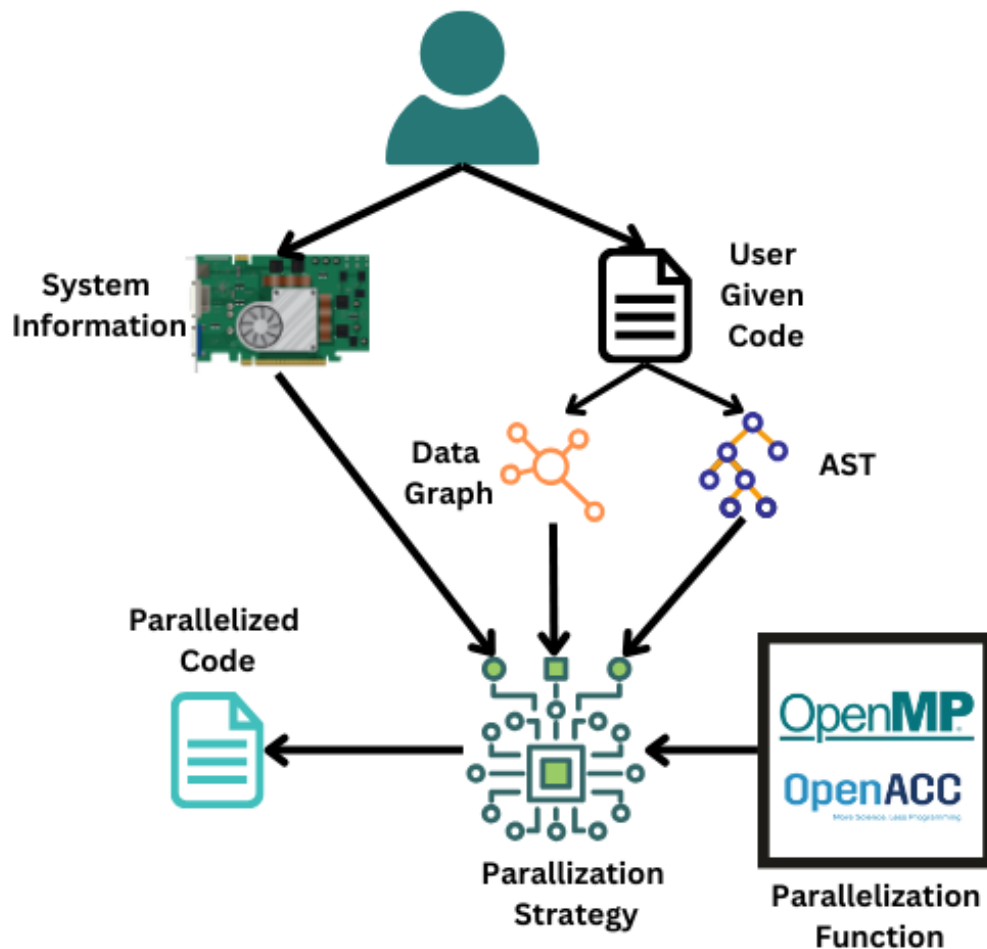


Figure 3.5: Level 2

3.2.2.4 Level 3 (Detailed Processes)

At this most detailed level, the diagram includes a comparison step. After the parallelized code is generated, it is compared with the user's initial code. This comparison evaluates the performance improvement and ensures that the parallelization process has achieved its intended goal. The system likely provides feedback on the optimization results, showing the user how much improvement was achieved or if further optimization is possible.

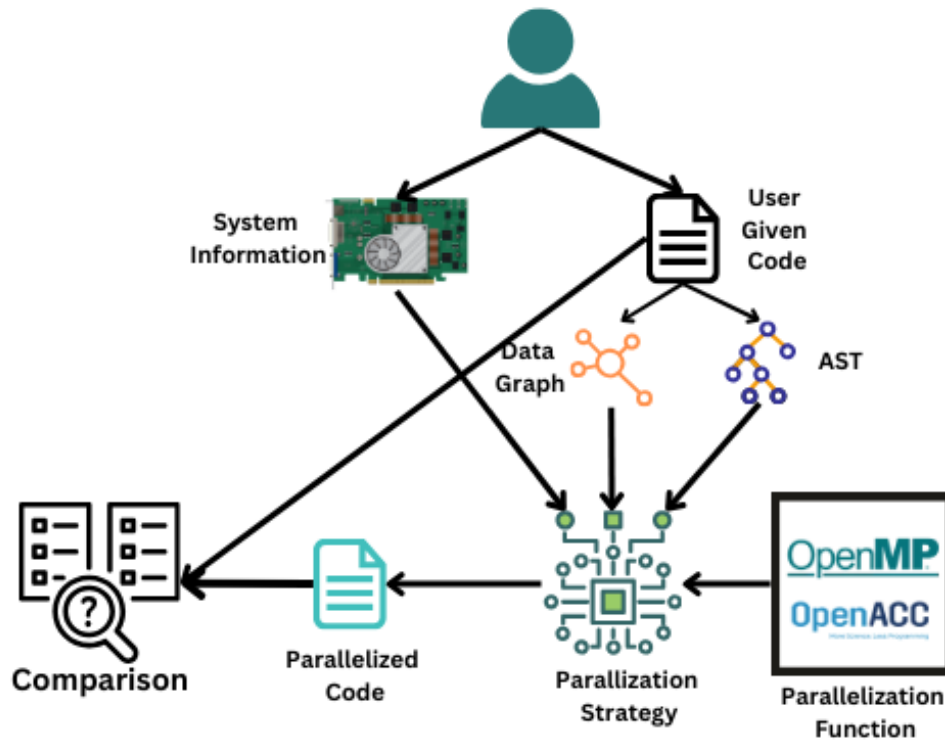


Figure 3.6: Level 3

Each level progressively adds more detail, providing a comprehensive view of how SpecBot processes code, from initial input to final optimized output.

3.2.3 System Level Sequence Diagrams

These diagrams illustrate the sequence of interactions between the user and the system for various modules like Code Analysis, Code Deployment, Parallelization Decision, Parinomo Engine, and Thread Optimization. They help in understanding the flow of operations and data exchange between different parts of the system.

3.2.3.1 Code Analysis Module

This diagram illustrates the interaction between the user and the system during code analysis. It shows the process of uploading code, generating the Abstract Syntax Tree (AST), and performing data dependency analysis. The diagram highlights how the system interacts with the user and tell the use if the code was parallelizable or not if wasn't it displays an appropriate message else it shows original and parallel code to the user.

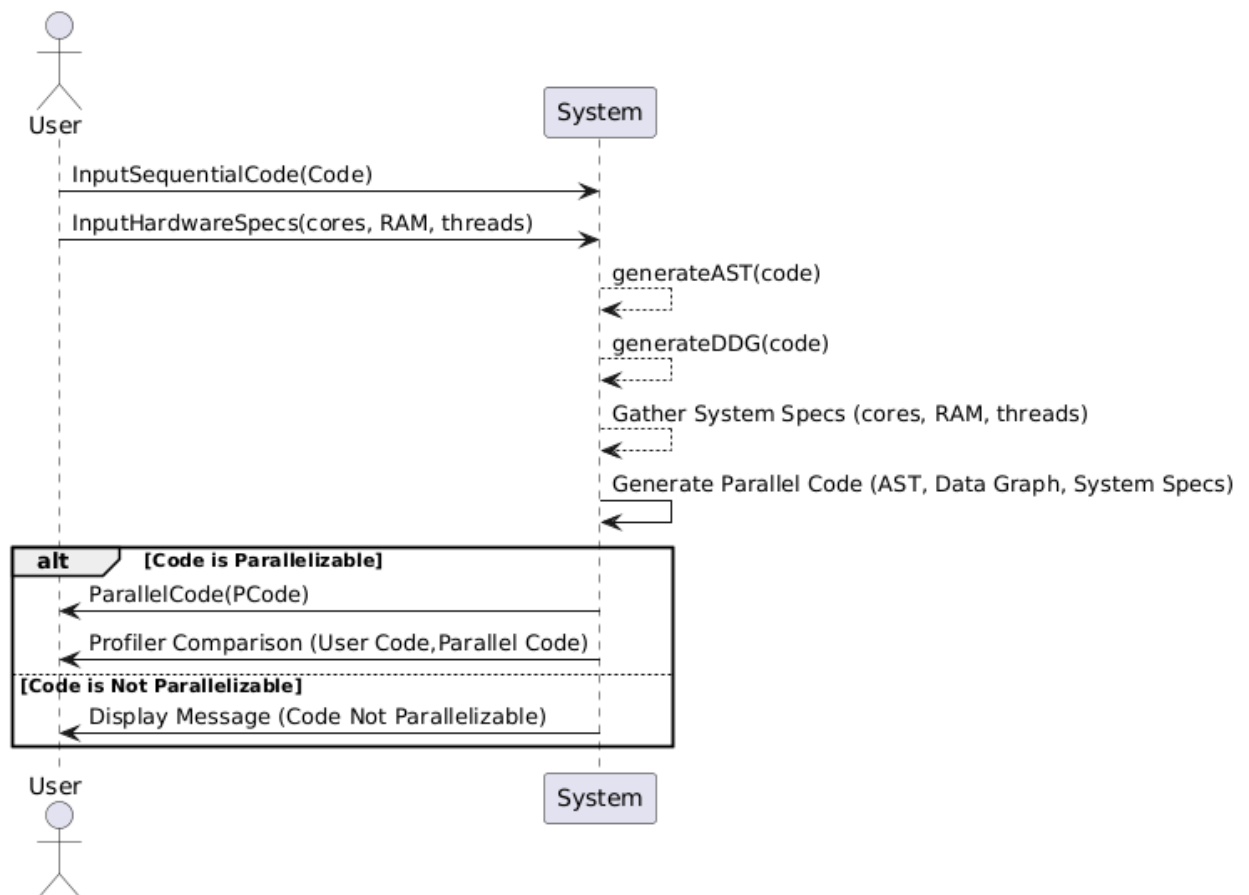


Figure 3.7: Code Analysis Module SSD

3.2.3.2 Code Deployment Module

The Code Deployment Module SSD demonstrates the sequence of actions involved in deploying the parallelized code. It shows how the system integrates the optimized code into the user's development environment, either through the Visual Studio Code extension or the web interface. A comparison of the original and parallelized code will also be shown to the user so that they may easily identify the changed parts of the code.

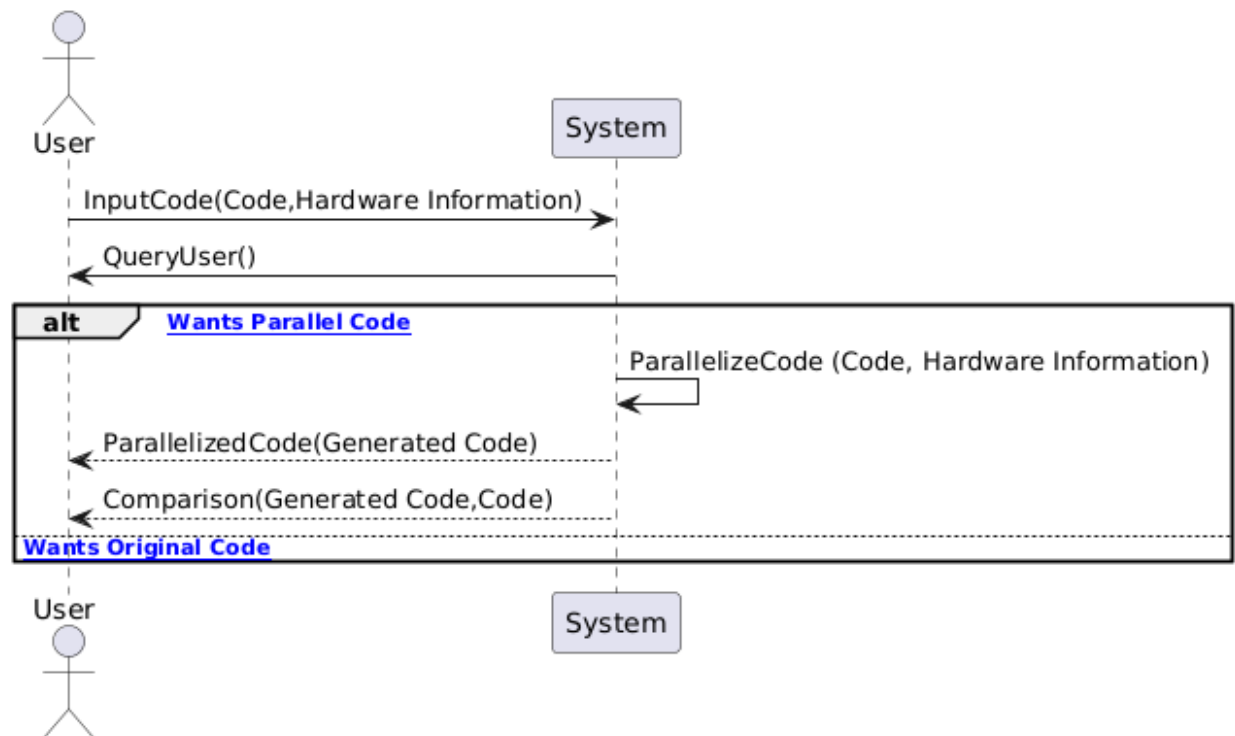


Figure 3.8: Code Deployment Module

3.2.3.3 Parallelization Decision Module

This SSD depicts the decision-making process for code parallelization. It shows how the system analyzes the code structure, hardware specifications, and data dependencies to determine the most suitable parallelization strategy. Each part of the code is split into blocks and checked which parts of the code may be parallelized and which should be left as serial.

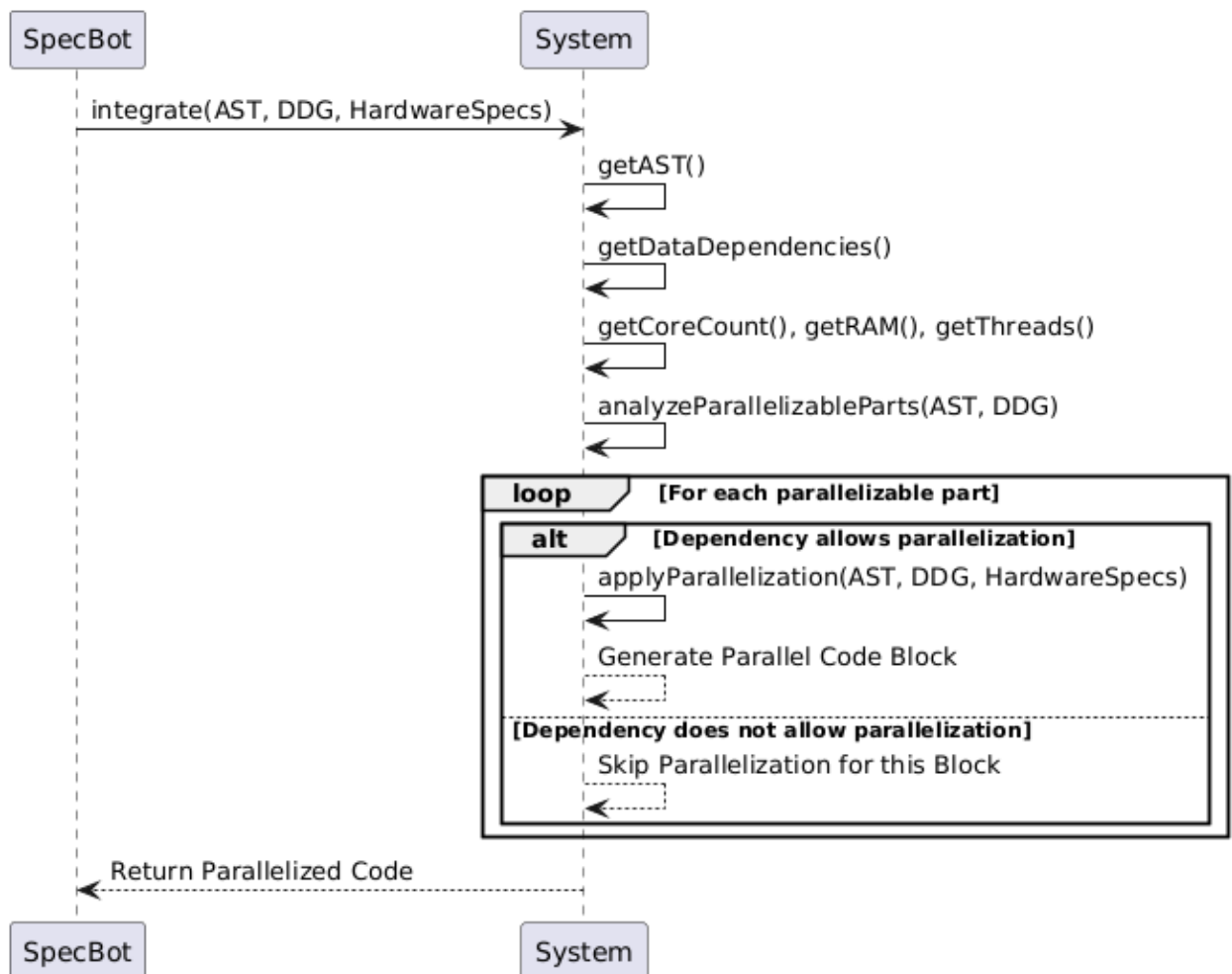


Figure 3.9: Parallelization Decision Module

3.2.3.4 Thread Optimization Module

This diagram represents the thread optimization process within SpecBot. It shows how the system determines optimal thread sizes for each parallelizable loop, implements dynamic thread adjustment during runtime, and prioritizes process threads in the system queue. The loops in the code will be parallelized using the most ideal number of threads which are selected using the data dependency graph.



Figure 3.10: Thread Optimization Module

3.2.3.5 Parinomo Engine Module

The Parinomo Engine Module SSD showcases the entire parallelization algorithm from taking the code and hardware information to generating the desired data structures. Splitting the code into different blocks and iterating through each block to identify which parts of the code can be parallelized and in case of loops which number of threads should be used.



Figure 3.11: Parinomo Engine Module

3.2.4 State Transition Diagram

A state diagram represents the condition of a system or a part of the system at specific moments in time. As a behavioral diagram, it illustrates the behavior through finite state transitions. In our project, both the website and the extension share the same state diagram, as they collect information from the user and send it to the backend server using the API.

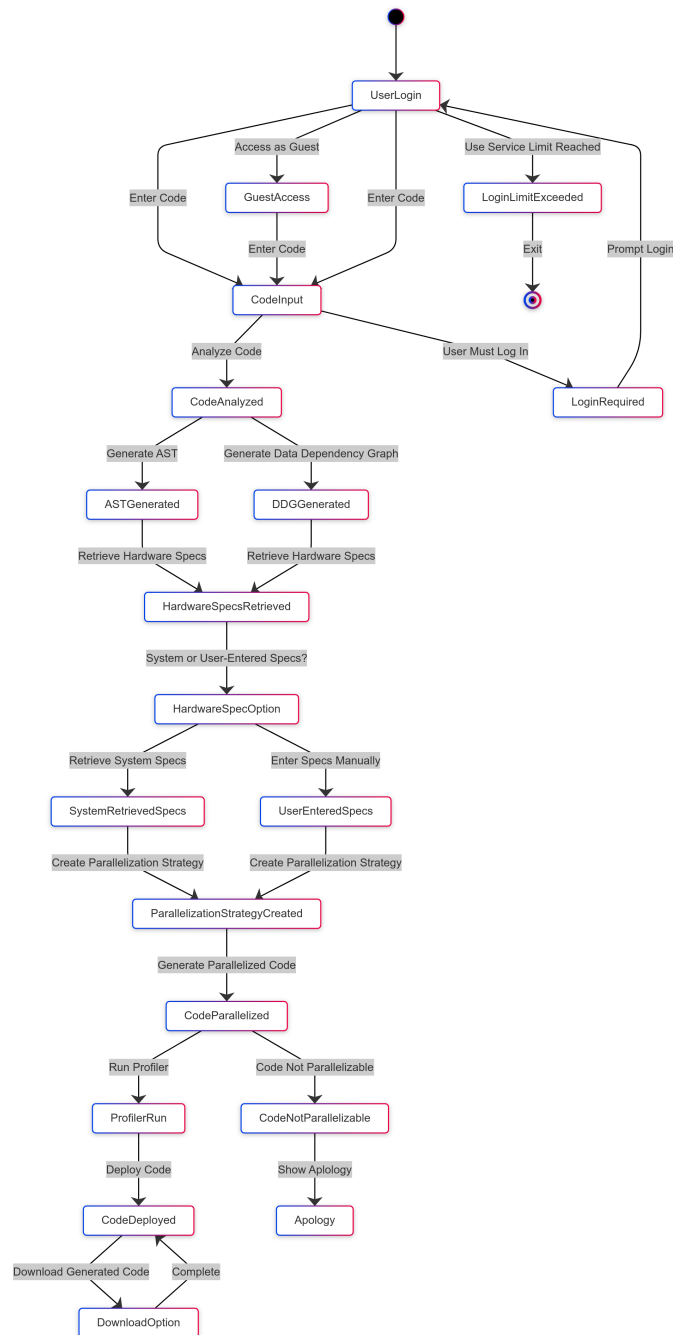


Figure 3.12: State Transition Diagram

3.3 Data Design

Our program uses the power of the Abstract Syntax Tree (AST) and a Data Dependency Graph to identify potential areas for parallelization. This makes it easier to insert the necessary OpenMP/OpenACC commands to achieve the desired results. The data structures used are explained as follow:

3.3.1 Data Structures Utilized

3.3.1.1 Abstract Syntax Tree

An Abstract Syntax Tree (AST) is a data structure used in computer science to represent the structure of a program or code snippet. By using the grammar of the programming language, each block of code can be easily identified and converted into a tree format. It allows nodes to be inserted into the tree more easier so it will allow use to convert the tree back into its code form.

3.3.1.2 Data Dependency Graph

A dependency graph represents the flow of information between attributes in a parse tree. It helps determine the evaluation order of these attributes. This allows us to decide how many threads are needed for loop parallelization, ensuring that variables are not overwritten and the data structure's integrity is maintained[2, 4].

3.3.2 Data Storage and Communication

3.3.2.1 JSON Schema

Communication between the website, the Visual Studio Code extension, and the backend server will happen through API calls. These calls will send and receive JSON files, which contain the necessary information to execute the parallelization strategy. The backend server will respond with a corresponding JSON file, providing instructions on how the frontend should react to the user[1].

1. **Frontend JSON Schema:** This schema shows how the data is sent to the server through the API call.
 - **ram_size:** Users machine RAM size or the defined size.
 - **thread_count:** The available number of threads present in the machine to be used.

- **cpu**: A string specifying whether the CPU is Intel or AMD. For validation.
- **code**: A string containing the user's original serial code.

```
1 {
2   "title": "Data to be sent to server",
3   "type": "object",
4   "properties": {
5     "ram_size": {
6       "type": "string",
7       "description": "The amount of RAM in the machine"
8     },
9     "thread_count": {
10      "type": "number",
11      "description": "The number of threads in the machine"
12    },
13    "cpu": {
14      "type": "string",
15      "description": "The CPU in the machine is it Intel or AMD"
16    },
17    "code": {
18      "type": "string",
19      "description": "The serial code given by the user"
20    }
21  },
22  "required": [
23    "ram_size",
24    "thread_count",
25    "cpu",
26    "code"
27  ]
28 }
```

Figure 3.13: Frontend Schema

2. **Response JSON Schema:** This schema shows the structure of the json returned to the user.

- **code**: A string containing the parallelized version of the user's code.
- **parallelized**: A boolean indicating whether the parallelization process was successful (true) or not (false).
- **language_supported**: A boolean confirming if the input code's language is supported by the system.

```
1 {
2   "title": "Response",
3   "type": "object",
4   "properties": {
5     "Code": {
6       "type": "string",
7       "description": "The code which has been Parallelized."
8     },
9     "Parallelized": {
10      "type": "boolean",
11      "description": "Tells if the code is parallelized or not."
12    },
13     "language Supported": {
14       "type": "boolean",
15       "description": "Tells if the given coding language is supported or not."
16     }
17   },
18   "required": [
19     "Code",
20     "Parallelized",
21     "language Supported"
22   ]
23 }
```

Figure 3.14: Backend Server Response

3.3.2.2 Authentication

The only data being stored are user credentials, specifically emails and passwords, managed securely through Firebase Authentication. Currently, we will not use emails in our project but will save them to keep track of the number of users. This ensures flexibility for future use, if needed, without requiring a traditional database.

By relying on these structures and lightweight communication protocols, we eliminate the need for large-scale data storage while maintaining efficient processing and optimization of user input code.

Bibliography

- [1]
- [2] Mar 2022.
- [3] Roshan Alwis. Parallel matrix multiplication [c][parallel processing], 2017.
- [4] GeeksforGeeks. Dependency graph in compiler design, Mar 2022.
- [5] K. Ikudome, G.C. Fox, A. Kolawa, and J.W. Flower. An automatic and symbolic parallelization system for distributed memory parallel computers. In *Proceedings of the Fifth Distributed Memory Computing Conference, 1990.*, volume 2, pages 1105–1114, 1990.
- [6] Tal Kadosh, Nadav Schneider, Niranjan Hasabnis, Timothy Mattson, Yuval Pinter, and Gal Oren. Advising openmp parallelization via a graph-based approach with transformers, 2023.
- [7] Bala Priya C. Abstract syntax tree (ast) - explained in plain english.
- [8] Michael Wolfe. Burying the openmp versus openacc hatchet, Jan 2019.